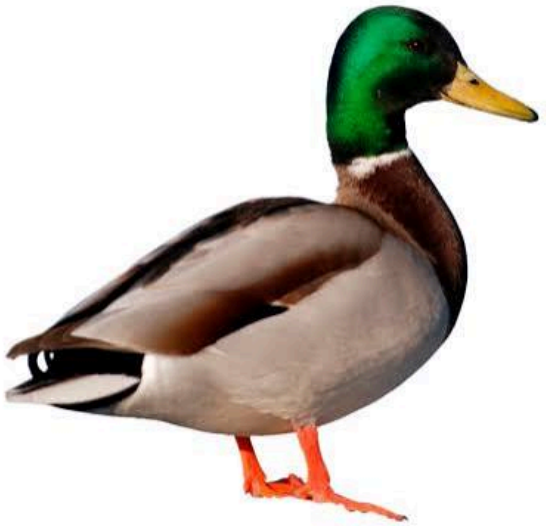
# Ciência de Dados Aplicada ao Direito I

## Aula 18

Como trabalhar na fase 2



Prof. Bruno da Cunha de Oliveira - 2026/2

# Instalar o Git

Insper

# Passo 1 do guia: instalar o Git

Abram o guia de instalação do seu sistema, no Blackboard, e façam só o passo 1. Os outros passos nós fazemos juntos, mais adiante.

| Sistema | O que o passo 1 pede |
|---|---|
| Windows | Baixar o instalador do Git, seguir as telas e reabrir o VS Code |
| Mac | Digitar `xcode-select --install` no terminal do VS Code e esperar o download |

Deu certo quando `git --version` responde com um número de versão.

# Quem usa IA, e para quê

Insper

# Como você usou IA neste mês

**EXERCÍCIO**

Qual destas frases descreve melhor o seu uso de IA nos últimos trinta dias?

**A.** Quase não usei.

**B.** Usei de vez em quando, para buscar ou para revisar um texto.

**C.** Usei toda semana, sempre em conversa: eu pergunto e ela responde.

**D.** Entreguei a um agente uma tarefa inteira, e ele a construiu sozinho.

# O funil do uso de IA

**Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.**

| | |
|---|---|
| **6 bilhões** | quase nenhum contato com modelos de linguagem |

# O funil do uso de IA

**Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.**

| | |
|---|---|
| **6 bilhões** | quase nenhum contato com modelos de linguagem |
| **1 a 2 bilhões** | uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever |

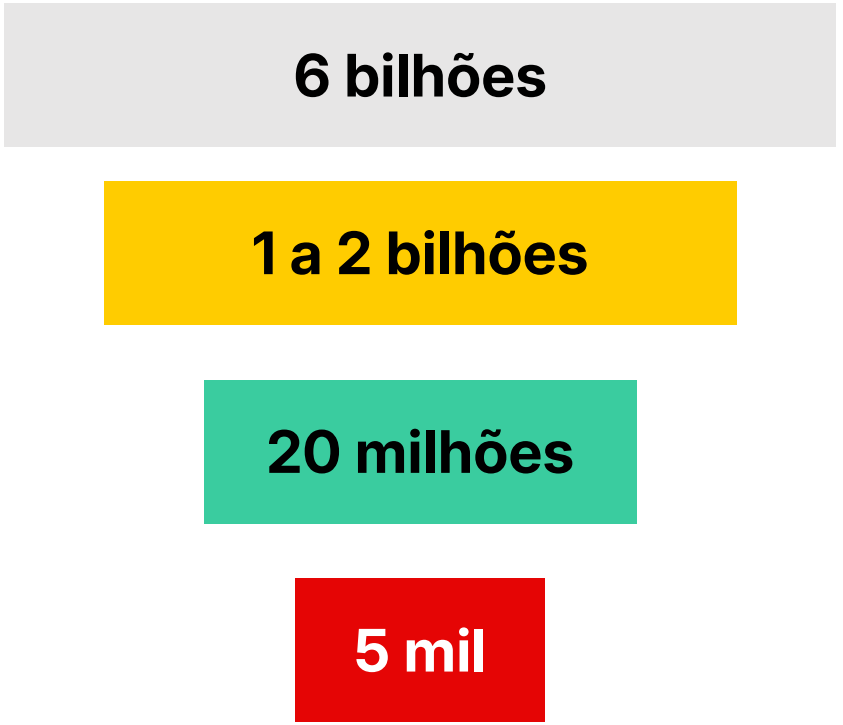
# O funil do uso de IA

**Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.**

| | |
|---|---|
| **6 bilhões** | quase nenhum contato com modelos de linguagem |
| **1 a 2 bilhões** | uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever |
| **20 milhões** | uso profissional de modelos de ponta em matemática e código: projetos de semanas feitos por agentes |

# O funil do uso de IA

**Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.**




quase nenhum contato com modelos de linguagem

uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever

uso profissional de modelos de ponta em matemática e código: projetos de semanas feitos por agentes

acesso a sistemas de ponta de uso interno, ainda não lançados

# Fase 2 da APS

Insper

# Da fase 1 para a fase 2

## Fase 1: vender a ideia

Pitch, site e relatório, para uma banca de advogados.

A IA ajudou vocês a pesquisar, escrever e montar o site.

# Da fase 1 para a fase 2

## Fase 1: vender a ideia

Pitch, site e relatório, para uma banca de advogados.

A IA ajudou vocês a pesquisar, escrever e montar o site.

## Fase 2: construir o produto

O produto que vocês venderam, funcionando, para uma banca de tecnologia.

Um assistente de código escreve o programa, e vocês dirigem: pedem, leem o que ele fez e decidem o que fica.

# De onde partir

## De um projeto aberto

Um fork é uma cópia de um repositório público na conta de vocês no GitHub.

A cópia já funciona, e o trabalho é mudar o que precisa mudar para virar o produto de vocês.

# De onde partir

## De um projeto aberto

Um fork é uma cópia de um repositório público na conta de vocês no GitHub.

A cópia já funciona, e o trabalho é mudar o que precisa mudar para virar o produto de vocês.

## Do zero

Vocês descrevem o produto e pedem ao assistente que o construa.

Não há nada para entender antes, e também não há nada funcionando no primeiro dia.

# Três formas de produto, e as bases de cada uma

| Como o advogado usa o produto | Bases do guia |
|---|---|
| Conversa com um agente que lê, cria e altera arquivos numa pasta | pi |
| Abre um site, envia um documento ou faz uma pergunta, e vê o resultado na tela | chatbot, da Vercel, ou começar do zero |
| Não pede nada: o produto trabalha sozinho e avisa | Hermes, NanoClaw ou OpenClaw |

O guia de bases traz uma página para cada uma. Nenhuma é obrigatória, e nenhuma é a recomendada.

# O que vale para todos os grupos

- Repositório público no GitHub, com o histórico de versões do produto.

# O que vale para todos os grupos

- Repositório público no GitHub, com o histórico de versões do produto.
- Um arquivo `README.md` que diz como rodar o produto.

# O que vale para todos os grupos

- Repositório público no GitHub, com o histórico de versões do produto.
- Um arquivo `README.md` que diz como rodar o produto.
- Relatório de uso de IA e de outras ferramentas, como na fase 1.

# O que vale para todos os grupos

- Repositório público no GitHub, com o histórico de versões do produto.
- Um arquivo `README.md` que diz como rodar o produto.
- Relatório de uso de IA e de outras ferramentas, como na fase 1.
- A linguagem de programação é livre.

# Quanto custa

Nenhum gasto é exigido.

Se o grupo decidir pagar por um modelo de linguagem ou por uma ferramenta, o relatório de uso de IA diz o que foi pago, quanto e por quê.

# Datas da fase 2

| Quando | O que acontece |
|---|---|
| ter 27/10 | A Temis apresenta como ganhou a fase 1. Eu explico como a banca avalia e como é a demonstração |
| ter 17/11 | Entrega, até o início da aula: endereço do repositório, titular e suplente. Quem investir entrega o contrato e o fato relevante |
| qua 18/11 | Sessão da fase 2, das 12h às 14h, com banca de pessoas de tecnologia |

# A linguagem de cada base

Quatro das cinco bases estão escritas numa linguagem que vocês não estudaram.

| Base | Linguagem |
|---|---|
| pi | TypeScript |
| chatbot, da Vercel | TypeScript |
| Hermes | Python |
| NanoClaw | TypeScript |
| OpenClaw | TypeScript |

# Linguagens de programação

Insper

# Nomes de linguagens de programação

**EXERCÍCIO**

Quantas destas oito palavras são nomes de linguagens de programação?

**Go Rust Lua Swift Ruby Julia Pato Nutella**

**A.** Duas

**B.** Quatro

**C.** Seis

**D.** Oito

# Seis linguagens, e uma delas brasileira

**RESPOSTA**

**C.** Go, Rust, Lua, Swift, Ruby e Julia são linguagens de programação em uso.

Lua foi criada em 1993 na PUC-Rio, que a mantém até hoje.

# Um programa em TypeScript

```typescript
function prazoEmDobro(dias: number): number {
  return dias * 2;
}

console.log(prazoEmDobro(15));
```

Este programa não está em Python. O que ele mostra na tela?

# A mesma função em Python e em TypeScript

**RESPOSTA**

O programa mostra `30`.

| Python | TypeScript | O que é |
|---|---|---|
| `def` | `function` | definir a função |
| indentação | `{ e }` | os limites do corpo |
| `print(...)` | `console.log(...)` | mostrar na tela |
| nada | `: number` | o tipo de cada valor |

# A versão do Python no seu computador

```
uv python list --only-installed
```

Rodem este comando no terminal do VS Code. Que versão do Python vocês têm? Ela é a mais nova que existe?

# Linguagem também tem versão

**RESPOSTA**

O Python ganha uma versão nova por ano. A 3.15 saiu em outubro de 2026.

```
uv self update
uv python install 3.15
uv python list --only-installed
```

A 3.15 fica instalada ao lado da versão que vocês já tinham. A pasta `cdad1` continua na versão antiga.

# Um pedido, doze arquivos alterados

Vocês pediram uma mudança ao assistente de código. Ele alterou doze arquivos, numa linguagem que vocês leem devagar, e o que funcionava ontem parou de funcionar.

O que vocês precisariam ter guardado para voltar ao que funcionava?

# Versões registradas

**RESPOSTA**

Uma cópia do projeto em cada ponto em que ele funcionava, com data e descrição, para a qual dê para voltar.

E um jeito de ver, linha a linha, o que mudou de uma cópia para a outra.

São as duas coisas que o Git faz.

# Git e GitHub

Insper

# Git e GitHub

**Git**

Um programa, no computador de vocês.

Registra as versões de uma pasta e deixa voltar a qualquer uma delas.

# Git e GitHub

## Git

Um programa, no computador de vocês.

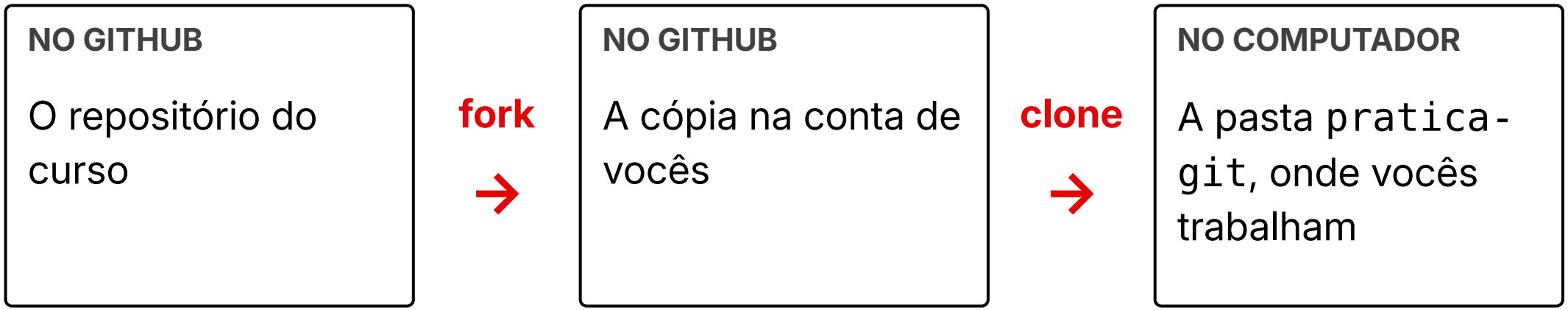
Registra as versões de uma pasta e deixa voltar a qualquer uma delas.

## GitHub

Um site.

Guarda uma cópia das versões, onde o grupo inteiro e a banca conseguem ver.

# Fork e clone


NO GITHUB
O repositório do curso
fork
NO GITHUB
A cópia na conta de vocês
clone
NO COMPUTADOR
A pasta pratica-git, onde vocês trabalham

# Passos 2, 3 e 4 do guia

| Passo | O que fazer |
|---|---|
| 2 | Criar a conta no GitHub |
| 3 | Dizer ao Git o seu nome e o seu e-mail |
| 4 | Fazer o fork do repositório de prática e o clone do seu fork |

Deu certo quando o VS Code mostra `prazos.py` na pasta `pratica-git`.

# Uma linha a mais em prazos.py

```
def prazo_em_dobro(dias):
    return dias * 2

print(prazo_em_dobro(15))
print(prazo_em_dobro(30))
```

A última linha acabou de ser escrita, e o arquivo foi salvo. O que falta para essa mudança virar uma versão a que dá para voltar, e para ela chegar ao GitHub?

# Registrar uma versão e enviar, passo a passo

Animação no navegador: `animacao-git.html`, cenário `registrar`.

# status, diff, add, commit e push

**RESPOSTA**

| Comando | O que faz |
|---|---|
| `git status` | diz o que mudou desde a última versão |
| `git diff` | mostra a mudança, linha a linha; a tecla q sai |
| `git add prazos.py` | escolhe o que entra na próxima versão |
| `git commit -m "..."` | registra a versão, com uma mensagem |
| `git push` | envia as versões novas ao GitHub |

# Uma versão sua no fork

No seu `prazos.py`, acrescentem uma linha que mostre o dobro de um prazo de 5 dias. Confiram a mudança, registrem a versão e enviem ao GitHub.

No primeiro `git push`, o login no GitHub é pelo navegador. Ninguém cria token nem digita senha no terminal.

Entreguem o endereço do seu fork pelo formulário "fork da aula 18", no Blackboard.

# A versão nova no GitHub

**RESPOSTA**

```
git diff
git add prazos.py
git commit -m "Mostra o prazo de 5 dias"
git push
```

Na página do fork, `prazos.py` mostra a linha nova, e o histórico passa de duas versões para três.

# Uma edição na função

```
def prazo_em_dobro(dias):
    return dias + 2

print(prazo_em_dobro(15))
print(prazo_em_dobro(30))
```

Alguém mexeu na segunda linha, e o programa roda sem erro. O que ele mostra?
Como voltar à versão registrada sem digitar a linha de novo?

# Voltar a uma versão registrada, passo a passo

Animação no navegador: `animacao-git.html`, cenário `voltar`.

# Desfazer com git restore

**RESPOSTA**

```
17
32
```

O programa responde 17 e 32, e não 30 e 60: nenhum erro avisa. `git diff` mostra a linha trocada.

```
git restore prazos.py
```

`git restore` descarta a mudança e traz o arquivo como estava na última versão registrada.

# O assistente de código

Insper

# Passo 6 do guia: Codex ou Claude

| Caminho | Onde roda | Que conta pede |
|---|---|---|
| Codex, aplicativo | num programa separado | conta do ChatGPT; a gratuita serve |
| Codex, extensão | dentro do VS Code | ChatGPT com plano Plus ou maior |
| Claude Code, extensão | dentro do VS Code | assinatura paga do Claude |

Escolham um, instalem e abram nele a pasta `pratica-git`.

# Um pedido ao assistente

Façam este pedido ao assistente:

> Acrescente ao `prazos.py` a função `dias_restantes(prazo, decorridos)`, que devolve quantos dias faltam. Depois mostre quantos dias restam de um prazo de 15 dias em dobro, passados 12 dias.

Antes de rodar o programa: que número tem que aparecer?

# A resposta antes do código

**RESPOSTA**

```
18
```

O dobro de 15 é 30, e 30 menos 12 é 18.

Vocês sabem a resposta antes de o assistente escrever uma linha, e é isso que permite conferir o que ele entregou.

# O que o assistente alterou

O assistente disse que terminou.

Com os comandos de hoje, como vocês descobrem o que ele alterou? E o que fazem se a mudança não servir?

# Ler o diff e decidir

**RESPOSTA**

| Pergunta | Comando |
|---|---|
| Que arquivos ele alterou? | `git status` |
| Que linhas ele alterou? | `git diff` |
| O programa mostra 18? | `uv run prazos.py` |
| A mudança fica | `git add` e `git commit` |
| A mudança não fica | `git restore` |

# O ciclo de um pedido ao assistente



# O programa do juscraper, de novo

Insper

# O programa do juscraper, de novo

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

Sem olhar as anotações: o que vocês já leem neste programa?

# O que já dá para ler

**RESPOSTA**

| No programa | Vocês leem como |
|---|---|
| `tjsp`, `dados`, `contagem` | objetos, cada um de uma classe |
| `.cjpg(...)`, `.head(5)` | métodos: o ponto pede uma ação |
| `dados.shape` | atributo: um dado, sem parênteses |
| `paginas=range(1, 11)` | argumento nomeado; páginas de 1 a 10 |

O `import`, os colchetes de `dados["assunto"]` e a classe desses objetos ficam para o próximo encontro.